
MuJoCo 学习笔记

Multi-Joint dynamics with Contact

魏舟

2026 年 2 月 7 日

Contents

1	Overview	1
1.1	引言 (Introduction)	1
1.1.1	历史与现状	1
1.1.2	核心架构	1
1.2	核心特性	2
1.2.1	广义坐标与现代接触动力学	2
1.2.2	凸优化接触模型	2
1.2.3	腱几何 (Tendon Geometry)	2
1.2.4	通用执行器模型	2
1.3	运行时架构	2
1.3.1	模型与数据的分离	2
1.3.2	可重构计算管线	2
1.4	模型元素 (Model Elements)	3
1.4.1	选项与统计 (Options and Statistics)	3
1.4.2	资产 (Assets)	3
1.4.3	建模语言 MJCF	3
1.5	代码与模型示例	3
1.5.1	简单的 MJCF 模型 (hello.xml)	3
1.5.2	基础模拟循环 (C/C++)	3
1.6	数学动力学 (补充)	4
2	计算理论 (Computation)	5
2.1	坐标空间与递归运动学	5
2.2	动力学方程的数学构成	5
2.2.1	核心矩阵深度详解	5
2.3	约束求解与优化 (Constraint Solver)	6
2.3.1	优化目标函数	6
2.3.2	摩擦锥模型	6
2.4	执行器与流水线阶段	6
2.4.1	执行器动力学 (Actuator Dynamics)	6
2.4.2	计算流水线详细阶段	6

2.5	解析性质：可微性 (Differentiability)	7
3	建模深度指南 (Modeling)	9
3.1	MJCF 设计哲学：为什么它比 URDF 更强？	9
3.2	编译器指令 (Compiler)：全局策略与自动化逻辑	9
3.2.1	坐标系策略：Local vs Global	9
3.2.2	自动物理计算：惯性、质量与中心	9
3.2.3	核心参数全解析表	10
3.2.4	进阶：Autofit (自动适配)	10
3.3	示例：一个高性能编译器的配置	10
3.4	参数间的相互作用：避坑指南	11
3.5	模型层级核心：Body, Joint 与 Geom	11
3.5.1	刚体 (Body)：纯粹的坐标系容器	11
3.5.2	关节 (Joint)：约束与自由度的精髓	11
3.5.3	几何体 (Geom)：外观、质量与碰撞	12
3.6	示例：标准动力学节点的配置	12
3.7	零基础避坑：坐标与层级的逻辑	13
3.8	碰撞管理 (Geoms) 与过滤逻辑	14
3.8.1	碰撞形状的选择	14
3.8.2	碰撞过滤：位运算详解 (Bitmask)	14
3.9	执行器与腱绳 (Actuators & Tendons)	14
3.9.1	执行器 (Actuators)：统一动力学模型	14
3.9.2	核心执行器类型速查	15
3.9.3	腱绳 (Tendons)：复杂约束的实现	15
3.10	示例：带默认属性与限位的多驱动配置	15
3.11	建模专家建议：数值稳定性调优	16
3.12	建模最佳实践与避坑指南	16
3.13	示例代码：完整模型结构	17
3.14	建模限制与优化建议	17
4	XML 引用指南 (XML Reference)	19
4.1	MJCF 根元素与全局配置	19
4.2	世界体与动力学组件	20
4.3	资产与驱动系统	20
4.4	传感器与约束 (Sensors and Equality)	21
4.5	高级设置	21
5	MJCF 实战：从零撰写规范模型	23
5.1	撰写流水线：四步建模法	23
5.2	第一步：建立“物理环境”与“默认样式”	23
5.3	第二步：构建层级化的“运动链”	24

5.4	第三步：赋予动力 (Actuators)	24
5.5	撰写时的核心原则 (Golden Rules)	24
5.5.1	1. 几何体优先原则	24
5.5.2	2. 命名规范化	24
5.5.3	3. 增量式开发	24
5.6	调试技巧：Viewer 是你最好的朋友	25
6	编程接口深度指南 (Programming Interface)	27
6.1	MuJoCo 编程哲学：设计与性能	27
6.2	核心数据结构详解	27
6.2.1	mjModel：静态模型定义	27
6.2.2	mjData：动态模拟数据	28
6.3	C API 计算管线：mj_step 的深度拆解	28
6.3.1	1. 位置级别函数 (Position-level)	28
6.3.2	2. 速度级别函数 (Velocity-level)	28
6.3.3	3. 力学与集成 (Force & Integration)	28
6.4	多线程并发与性能优化	29
6.4.1	线程安全策略	29
6.4.2	内存池管理 (Arena Memory)	29
6.5	超长程序示例：高频率采样控制循环	29
6.6	常用 API 函数分类速查	30
6.7	编程中的常见陷阱与调试技巧	31
7	Python 编程接口 (Python Bindings)	33
7.1	为什么选择 Python 接口？	33
7.2	基础核心：加载与步进	33
7.2.1	加载模型	33
7.3	数据交互：读取与控制	34
7.3.1	状态读取 (Getter)	34
7.3.2	控制输入 (Setter)	34
7.4	高级特性：命名访问 (Named Access)	34
7.5	零基础实战：手把手构建一个仿真环境	34
7.6	可视化渲染 (Rendering)	35
7.7	数值稳定性与性能调优 (给新手的建议)	36
7.8	常用 API 速查表 (Python 版)	36
8	MJX：GPU 加速仿真 (MuJoCo on JAX)	37
8.1	什么是 MJX？——从“单兵作战”到“军团冲锋”	37
8.2	MJX 的核心概念：向量化 (Vectorization)	37
8.3	快速入门：MJX 编程三部曲	37
8.4	深度解析：MJX 与原生 MuJoCo 的差异	38

8.5 零基础实战：大规模并行采样 38

8.6 给新手的避坑指南 39

8.6.1 1. “冷启动”等待 39

8.6.2 2. 静态与动态的界限 39

8.6.3 3. 内存显存限制 39

8.7 MJX 的典型应用场景 39

Chapter 1

Overview

1.1 引言 (Introduction)

MuJoCo 是 *Multi-Joint dynamics with Contact* (带接触的多关节动力学) 的缩写。它是一个通用目的物理引擎，旨在促进机器人、生物力学、图形与动画、机器学习等领域的研发。

1.1.1 历史与现状

MuJoCo 最初由 Roboti LLC 开发，2021 年 10 月由 DeepMind 收购并免费开放，2022 年 5 月正式开源。其代码库目前托管在 GitHub 的 `google-deepmind/mujoco` 仓库。

1.1.2 核心架构

MuJoCo 是一个带有 C API 的 C/C++ 库。其运行时的模拟模块通过预分配的低级数据结构实现性能最大化。用户通常使用 **MJCF** (一种 XML 场景描述语言) 定义模型，同时也支持加载 **URDF** 文件。此外，它包含一个基于 OpenGL 渲染的原生交互式可视化 GUI。

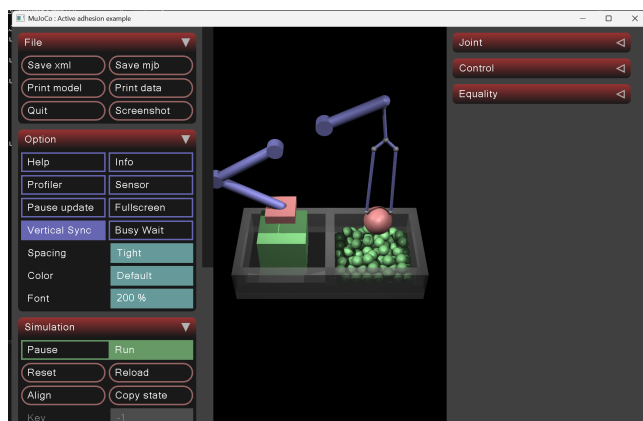


Figure 1.1: MuJoCo 图形用户界面 (GUI) 示意图

1.2 核心特性

1.2.1 广义坐标与现代接触动力学

MuJoCo 将传统机器人学引擎的递归算法（广义坐标）与游戏引擎的优化驱动接触动力学结合。它在保证运动学结构稳定性的同时，解决了复杂的接触冲突。

1.2.2 凸优化接触模型

MuJoCo 的接触模型基于不同的物理公式，可简化为凸优化问题。

- 支持软接触、约束。
- 具有独特的解析可逆性，极大方便了逆动力学分析和控制应用。
- 统一处理：摩擦接触、滑轮限制、干摩擦、等式约束。

1.2.3 腱几何 (Tendon Geometry)

支持模拟三维空间的腱绳。这些腱绳遵循最小路径长度原则，可以受到绕过点（via-point）和包裹（wrapping）约束。

1.2.4 通用执行器模型

通过抽象的执行器模型，MuJoCo 可以统一模拟电机、气动/液压缸、PD 控制器以及复杂的生物肌肉。

1.3 运行时架构

1.3.1 模型与数据的分离

MuJoCo 运行时严格区分了两个核心数据结构：

数据分离设计

- **mjModel**: 包含模型描述，通常保持不变。
- **mjData**: 包含所有动态变量和中间计算结果。

这种设计支持多线程采样和有限差分计算，因为可以在多个 **mjData** 实例上同时应用同一个 **mjModel**。

1.3.2 可重构计算管线

MuJoCo 提供了大量的 *Flags*，允许用户在不改变算法代码的情况下，通过开关特定模块（如忽略重力、关闭碰撞等）来重新配置计算管线。

1.4 模型元素 (Model Elements)

1.4.1 选项与统计 (Options and Statistics)

- **mjOption:** 物理参数、算法选择。
- **mjVisual:** 渲染属性。
- **mjStatistic:** 编译器计算的统计量（如平均质量、空间范围）。

1.4.2 资产 (Assets)

资产是模型引用的外部资源：

- **Mesh:** 支持 OBJ 和二进制 STL。
- **Skin:** 仅用于渲染的变形皮肤，增加视觉真实感。
- **Height field:** 从 PNG 或二进制文件加载的地形数据。
- **Texture & Material:** 控制颜色、反光、纹理映射。

1.4.3 建模语言 MJCF

MJCF 具有类似 CSS 的默认设置机制，这使得模型文件极其简洁易读。

1.5 代码与模型示例

1.5.1 简单的 MJCF 模型 (hello.xml)

```
1 <mujoco>
2   <worldbody>
3     <light diffuse=".5 .5 .5" pos="0 0 3" dir="0 0 -1"/>
4     <geom type="plane" size="1 1 0.1" rgba=".9 0 0 1"/>
5     <body pos="0 0 1">
6       <joint type="free"/>
7       <geom type="box" size=".1 .2 .3" rgba="0 .9 0 1"/>
8     </body>
9   </worldbody>
10 </mujoco>
```

1.5.2 基础模拟循环 (C/C++)

```
1 #include "mujoco.h"
2
3 int main(void) {
4   char error[1000];
```

```
5   mjModel* m = mj_loadXML("hello.xml", NULL, error, 1000);
6   mjData* d = mj_makeData(m);
7
8   while (d->time < 10) {
9       mj_step(m, d); // 核心计算步进
10  }
11
12  mj_deleteData(d);
13  mj_deleteModel(m);
14  return 0;
15 }
```

1.6 数学动力学 (补充)

MuJoCo 的前向动力学可以概括为:

$$\dot{\mathbf{v}} = \mathbf{M}(\mathbf{q})^{-1} (\boldsymbol{\tau} + \mathbf{J}(\mathbf{q})^T \mathbf{f}_{contact} - \mathbf{c}(\mathbf{q}, \mathbf{v})) \quad (1.1)$$

其中 $\mathbf{M}$ 是惯性矩阵, $\mathbf{c}$ 是偏置力 (重力、科氏力等), $\mathbf{f}_{contact}$ 是通过凸优化求解的接触力。

Chapter 2

计算理论 (Computation)

2.1 坐标空间与递归运动学

MuJoCo 的计算效率源于其在 **最小坐标系** 下的递归实现。

1. 位置与速度的表示 系统的状态由位形向量 $\mathbf{q}$ (位置) 和切空间向量 $\mathbf{v}$ (速度) 描述:

- 广义位置 $\mathbf{q}$ (n_q 维): 包含自由关节 (free joint) 时, 使用单位四元数表示旋转。此时 $n_q > n_v$, 四元数在模拟中必须保持归一化以确保物理合法性。
- 广义速度 $\mathbf{v}$ (n_v 维): 直接对应自由度 (DOF)。在物理上, 它是位形流形上的切矢量, 包含线速度与角速度。

2. 递归运动学 (Recursive Kinematics) MuJoCo 通过树状拓扑递归计算空间位姿。对于每个刚体 i , 其世界坐标系下的变换 T_i 是父节点变换 $T_{parent(i)}$ 与关节局部变换 $T_{joint(i)}$ 的复合函数。

2.2 动力学方程的数学构成

MuJoCo 求解的核心方程为:

$$\mathbf{M}(\mathbf{q})\dot{\mathbf{v}} + \mathbf{c}(\mathbf{q}, \mathbf{v}) = \boldsymbol{\tau}_{app} + \mathbf{J}(\mathbf{q})^T \mathbf{f} \quad (2.1)$$

2.2.1 核心矩阵深度详解

A. 惯性矩阵 $\mathbf{M}(\mathbf{q})$ 与 CRBA 算法 惯性矩阵描述了关节空间的质量分布, 是正定且对称的。

- **CRBA 算法**: 使用复合刚体算法 (Composite Rigid Body Algorithm)。它将当前关节及其所有后继节点视为一个复合体, 从叶子节点向根节点递归计算。
- **稀疏性利用**: 利用多体系统的树状分支特性, CRBA 仅计算非零项, 将复杂度控制在 $O(n^2)$ 。

B. 偏置力 $c(q, v)$ 与 RNEA 算法 偏置力涵盖了除加速度外的所有项: $c = C(q, v)v + g(q)$ 。

- **物理构成:** 包含科氏力、离心力以及引力矩。
- **RNEA 算法:** 采用递归牛顿-欧拉算法。通过“前向传递速度加速度”和“后向传递力与力矩”两次扫描, 实现 $O(n)$ 复杂度的计算。

C. 雅可比矩阵 $J(q)$ 与 Delassus 算子 雅可比矩阵是坐标系变换的微分映射。

- **对偶映射:** $\dot{x} = Jv$ (速度映射); $\tau = J^T f$ (力矩映射)。
- **Delassus 算子:** 定义为 $A = JM^{-1}J^T$ 。它描述了在接触点施加冲量时, 接触点瞬时加速度的响应, 是约束求解的核心输入。

2.3 约束求解与优化 (Constraint Solver)

MuJoCo 将接触力的求解转化为一个 ** 凸优化问题 **, 而非传统的线性互补问题 (LCP)。

2.3.1 优化目标函数

约束力 f 通过最小化以下代价函数获得:

$$\min_f \mathcal{L}(f) = \frac{1}{2} f^T (A + R) f + f^T a_{ref} \quad \text{s.t.} \quad f \in \mathcal{K} \quad (2.2)$$

其中 R 是用于软约束的正则化矩阵, $\mathcal{K}$ 是摩擦锥约束集。

2.3.2 摩擦锥模型

- **金字塔形 (Pyramidal):** 线性化近似, 计算速度快。
- **椭圆形 (Elliptic):** 精确符合库仑摩擦定律, 计算量略大, 但旋转一致性好。

2.4 执行器与流水线阶段

2.4.1 执行器动力学 (Actuator Dynamics)

MuJoCo 执行器支持带内部状态的动力学模型。例如生物肌肉模型包含激活度状态 act , 使得系统动力学从二阶扩展至三阶响应。

2.4.2 计算流水线详细阶段

MuJoCo 的 `mj_step` 严格遵循以下计算链:

mj_step 执行管线

1. 前向运动学：更新所有物体的世界位姿。
2. 碰撞检测：搜索重叠，生成接触流形 (Contacts)。
3. 动力学准备：执行 RNEA 和 CRBA，构造惯性矩阵与偏置力。
4. 约束构造：计算雅可比 J 、算子 A 及参考加速度 a_{ref} 。
5. 优化求解：调用 PGS/CG 求解器获取约束力 f 。
6. 数值集成：应用加速度 $\ddot{v}$ 更新状态 (q, v) 。

2.5 解析性质：可微性 (Differentiability)

不同于大多数具有开关特性的模拟器，MuJoCo 的软约束模型使得动力学函数对状态和控制输入通常是 ** 连续可微 ** 的。这一特性使得 MuJoCo 成为系统辨识、轨迹优化（如 iLQR）以及强化学习中基于梯度的策略改进（如 PPO/SAC 的动力学建模）的理想工具。

Chapter 3

建模深度指南 (Modeling)

3.1 MJCF 设计哲学：为什么它比 URDF 更强？

MuJoCo 的 MJCF 并非简单的 XML 描述，它是一个“编译型”格式。其核心优势在于：

- **整体性**：它可以自动计算复杂的质量分布和碰撞外壳。
- **类 CSS 机制**：通过 `<default>` 标签，可以用极少的代码描述复杂的机器人。

3.2 编译器指令 (Compiler)：全局策略与自动化逻辑

`<compiler>` 标签是 MJCF 的“大脑”，它告诉 MuJoCo 如何解读你的模型，并决定了哪些物理量需要自动计算。

3.2.1 坐标系策略：Local vs Global

这是建模的第一步选择。它决定了你在 `pos`（位置）属性中填写的数字到底是什么意思。

- **local (局部坐标, 墙裂推荐)**：每一个 `<body>` 的位置都是相对于它的 **** 父节点 **** 定义的。
优点：如果你移动了躯干，胳膊会自动跟着走，不需要重新计算胳膊的世界坐标。这符合机器人学的动力学链逻辑。
- **global (全局坐标)**：所有物体的坐标都是相对于世界原点 (0,0,0)。缺点：极难维护。如果你把底座挪了 1cm，你必须手动修改后续所有关节的坐标。

3.2.2 自动物理计算：惯性、质量与中心

对于零基础用户，手动计算一个复杂零件的转动惯量矩阵是极其痛苦且易错的。MuJoCo 提供了全自动方案。

1. **inertiafromgeom** (从几何体推算惯性) 这是编译器最重要的功能。

- **inertiafromgeom="true"**：你只需要定义物体的形状 (Geom) 和密度 (Density)。编译器会自动计算出：

- 1. **Mass (质量)**: 体积 $\times$ 密度。
- 2. **Inertia (转动惯量)**: 描述物体旋转时的惯性。
- 3. **Center of Mass (质心)**: 物体的平衡点。

2. **inertiagrouprange (惯性计算范围)** 如果你只想根据某些特定层级的几何体计算惯性（比如忽略视觉装饰物），可以使用此参数。

3.2.3 核心参数全解析表

Table 3.1: Compiler 深度参数词典

参数	功能深度解析
angle	degree （度）或 radian （弧度）。决定了 range 和 euler 属性的单位。建议统一使用一种。
meshdir	定义 Mesh 文件夹。例如设置为 "assets/" ，则 file="arm.stl" 会被解析为 assets/arm.stl 。
texturedir	同理，定义贴图文件夹。
boundmass	数值保险 。如果自动计算出的质量小于该值（如 $1e-6$ ），则强制设为此值。防止极小质量导致除以零爆炸。
boundinertia	惯量保险 。防止旋转动力学计算时出现数值不稳定的极小值。
discardvisual	如果设为 true ，编译器会忽略所有 group="4" （默认视觉组）及以上的几何体，仅保留物理碰撞体，显著提升复杂场景的计算速度。
settle	这是一个“一键归位”功能。如果你的初始位置有轻微重叠，它会在加载瞬间尝试让物体分开。

3.2.4 进阶：Autofit (自动适配)

MuJoCo 编译器还有一个隐藏技能：**自动缩放 (Autofit)**。如果你提供了一个巨大的 STL 网格，但想让它适应一个微小的胶囊体，可以通过设置编译器参数让 MuJoCo 自动调整网格的缩放比例 (Scale)，使其外壳紧贴物理碰撞体。

3.3 示例：一个高性能编译器的配置

```
1 <mujoco>
2   <compiler
3     coordinate="local"      angle="degree"      inertiafromgeom="true"
4     meshdir="assets/meshes" discardvisual="false" boundmass="0.0001" />
5 </mujoco>
```


Listing 3.1: 推荐的工业级编译器配置

3.4 参数间的相互作用：避坑指南

- 单位冲突：**如果你在 `compiler` 里写了 `angle="degree"`，但在引用外部插件时使用了弧度，MuJoCo 会报错或让你的机器人“抽风”。
- 密度与质量：**一旦开启 `inertiafromgeom="true"`，请确保你的 `<geom>` 中定义了 `density`（默认 1000，即水的密度）。如果某个物体的材质特别重（如钢），记得单独修改该 `geom` 的密度。
- 精度丢失：**对于微型机器人（毫米级），请务必调小 `boundmass` 和 `boundinertia`，否则编译器会把你的微型零件强行变重，导致仿真不准。

3.5 模型层级核心：Body, Joint 与 Geom

在 MuJoCo 中，机器人被描述为一个树状结构。理解这三个核心组件的层级嵌套逻辑是建模的灵魂。我们可以将其类比为：**Body** 是骨架坐标系，**Joint** 是筋腱约束，而 **Geom** 是长在骨架上的血肉。

3.5.1 刚体 (Body)：纯粹的坐标系容器

`<body>` 本身不具有物理实体，它是一个 **** 虚拟的局部坐标系偏移量 ****。

- pos & quat：**定义了该 Body 坐标系相对于父节点坐标系的相对位置和旋转。
- 层级继承：**如果父 Body 发生移动，其下嵌套的所有子 Body、Joint 和 Geom 都会作为一个整体进行同步空间变换。
- 惯性 (Inertial)：**Body 承载了该节点的动力学属性（质量 m 、质心 com 、惯量张量 I ）。通常由编译器根据内部的 Geom 自动推算。

3.5.2 关节 (Joint)：约束与自由度的精髓

关节不是实实体，而是 **** 允许 **** 子 Body 相对于父 Body 产生何种运动的“许可协议”。

核心参数深度解析

Table 3.2: Joint 动力学参数详解

参数	功能与物理意义
type	hinge: 1 轴旋转； slide: 1 轴平移； ball: 3 轴旋转； free: 6 自由度浮动。

armature	转子惯量。 模拟电机转子自身的惯性。在高频碰撞中，设置微小值（如 0.01）能极大提升数值稳定性，防止系统“爆炸”。
stiffness	弹簧刚度 k。 计算公式： $\tau = -k(q - q_{ref})$ 。用于模拟被动回弹力。
damping	阻尼系数 d。 计算公式： $\tau = -d\dot{q}$ 。模拟摩擦损耗，是系统能量的“消散器”。
frictionloss	干摩擦。 不同于阻尼，这是恒定的静摩擦力，力矩必须超过此值关节才会转动。

3.5.3 几何体 (Geom): 外观、质量与碰撞

`<geom>` 是仿真中唯一能被“看见”且能产生“碰撞力”的实体。

关键属性分析

- **type (形状选择):**
 - **capsule (胶囊体):** ** 仿真之王 **。其碰撞算法有解析解，接触力平滑，是建模机器人肢体的首选。
 - **mesh:** 支持外部 3D 模型。MuJoCo 会自动生成其“凸包” (Convex Hull) 用于计算。
- **density (密度):** 配合 `compiler` 使用，默认 1000（水的密度）。编译器根据体积 $\times$ 密度自动赋予 Body 质量。
- **solimp & solref:** 控制碰撞的“软硬”。
 - **solref:** 前两个参数分别是（时间常数，阻尼比）。如果物体落地后疯狂反弹，通常需要调大时间常数。

3.6 示例：标准动力学节点的配置

```

1 <mujoco model="example">
2   <default>
3     <geom rgba=".8 .6 .4 1"/>
4   </default>
5
6   <asset>
7     <texture type="skybox" builtin="gradient" rgb1="1 1 1" rgb2=".6 .8 1" width=
      "256" height="256"/>
8   </asset>
9
10  <worldbody>
11    <light pos="0 1 1" dir="0 -1 -1" diffuse="1 1 1"/>

```

```

12 <body pos="0 0 1">
13   <joint type="ball"/>
14   <geom type="capsule" size="0.06" fromto="0 0 0 0 0 -.4"/>
15   <body pos="0 0 -0.4">
16     <joint axis="0 1 0"/>
17     <joint axis="1 0 0"/>
18     <geom type="capsule" size="0.04" fromto="0 0 0 .3 0 0"/>
19     <body pos=".3 0 0">
20       <joint axis="0 1 0"/>
21       <joint axis="0 0 1"/>
22       <geom pos=".1 0 0" size="0.1 0.08 0.02" type="ellipsoid"/>
23       <site name="end1" pos="0.2 0 0" size="0.01"/>
24     </body>
25   </body>
26 </body>
27
28 <body pos="0.3 0 0.1">
29   <joint type="free"/>
30   <geom size="0.07 0.1" type="cylinder"/>
31   <site name="end2" pos="0 0 0.1" size="0.01"/>
32 </body>
33 </worldbody>
34
35 <tendon>
36   <spatial limited="true" range="0 0.6" width="0.005">
37     <site site="end1"/>
38     <site site="end2"/>
39   </spatial>
40 </tendon>
41 </mujoco>

```

Listing 3.2: Body-Joint-Geom 嵌套示例

3.7 零基础避坑：坐标与层级的逻辑

1. **位置的相对性**：在 `coordinate="local"`（默认）模式下，`<body>` 的 `pos` 是相对于其直接父节点的。如果你移动了父节点，子节点会成比例跟随。
2. **无关节即焊死**：如果一个 Body 下没有任何 Joint 定义，它会与其父节点 **** 焊接 **** 在一起，失去运动自由度。
3. **Free Joint 必须放在最顶层**：如果你想模拟一个自由跌落的盒子，必须在盒子的 Body 下第一个定义 `type="free"` 的关节，否则它会静止在空中。

3.8 碰撞管理 (Geoms) 与过滤逻辑

3.8.1 碰撞形状的选择

MuJoCo 采用“分析解”计算碰撞。

- **Capsule (胶囊体)**: 计算最快, 最推荐用于机器人肢体。
- **Mesh**: MuJoCo 会自动为 Mesh 生成 **Convex Hull (凸包)**。注意: MuJoCo 不支持原生的凹面碰撞。

3.8.2 碰撞过滤: 位运算详解 (Bitmask)

这是新手最容易困惑的地方。MuJoCo 使用两个 32 位整数来判断两个 geom A 和 B 是否碰撞:

碰撞触发公式

触发碰撞的条件: $(A.\text{contype} \& B.\text{conaffinity}) \parallel (B.\text{contype} \& A.\text{conaffinity})$

- **contype**: 我是谁 (属于哪个位)。
- **conaffinity**: 我想撞谁 (关注哪个位)。

3.9 执行器与腱绳 (Actuators & Tendons)

执行器是连接控制算法与物理世界的桥梁。MuJoCo 提供了一个统一的数学框架来描述从简单的直流电机到复杂的生物肌肉。

3.9.1 执行器 (Actuators): 统一动力学模型

MuJoCo 的执行器力矩输出遵循一个核心公式:

$$\text{Force} = (\text{Gain} \cdot \text{ctrl}) + \text{Bias} \quad (3.1)$$

这个模型将执行器拆解为三个独立配置的阶段:

1. Transmission (传动机制) 定义力 (或力矩) 作用在哪个自由度上。

- **Joint**: 最常见, 直接作用于滑动或转动关节。
- **Tendon**: 通过腱绳传递, 可以实现跨关节的拉力。
- **Site**: 直接在某个空间点施加线性力。

2. Gain (增益阶段) 决定了输入信号 `ctrl` 如何转换为原始动力。对于电机 (Motor), Gain 为常数 (即力矩常数); 对于伺服器 (Position Servo), Gain 对应比例增益 k_p 。

3. Bias (偏置阶段) 定义了与输入信号无关的附加力。

- 位置伺服器：偏置项通常定义为 $-k_p \cdot \text{position}$ 。
- 组合效果：当 Gain 和 Bias 同时作用时，产生 $k_p(\text{ctrl} - \text{position})$ ，这正是标准的比例控制逻辑。

3.9.2 核心执行器类型速查

Table 3.3: 执行器类型及其物理特性

类型	深度特性解析
motor	直接输出广义力。参数 <code>gear</code> 定义了减速比或力矩杠杆，公式为 $F = \text{ctrl} \cdot \text{gear}$ 。
position	实现位置闭环。通过 <code>kp</code> 调节控制强度。若系统产生高频振荡，必须通过关节阻尼（Damping）进行抑制。
velocity	实现速度闭环。利用 <code>kv</code> 维持恒定转速，常用于移动机器人的轮子驱动。
muscle	生物力学专用。包含复杂的长度-力（Length-Force）和速度-力（Velocity-Force）曲线，模拟真实肌肉的非线性特性。

3.9.3 腱绳 (Tendons): 复杂约束的实现

腱绳用于描述非刚性的连接逻辑，在欠驱动机器人和生物仿真中至关重要。

1. Fixed Tendon (固定联动) 用于定义关节间的线性耦合。物理示例：机械抓手的联动。通过定义 $q_{\text{finger2}} = c \cdot q_{\text{finger1}}$ ，可以让多个关节仅由一个执行器驱动。

2. Spatial Tendon (空间腱) 模拟穿过滑轮或绕过骨骼的绳索。

- Viapoints: 通过 `<site>` 标签定义绳索必须经过的中间点。
- Wrapping: 支持绕过几何体（如圆柱体或球体）。当关节转动时，腱绳的路径长度和力矩臂会动态改变，这能高精度模拟人体肌肉（如股四头肌绕过髌骨）。

3.10 示例：带默认属性与限位的多驱动配置

以下示例演示了如何通过 `<default>` 减少冗余代码，并实现一个具备物理保护的伺服系统。

```
1 <mujoco>
2   <default>
3     <position ctrllimited="true" ctrlrange="-1.5 1.5" kp="100"/>
4   </default>
5
```

```

6    <worldbody>
7      <body name="arm" pos="0 0 0">
8        <joint name="hinge1" type="hinge" axis="0 1 0" damping="0.5"/>
9        <geom type="capsule" size="0.03" fromto="0 0 0 0 0 0.4"/>
10       <site name="anchor" pos="0 0 0.4"/> </body>
11    </worldbody>
12
13    <tendon>
14      <spatial name="pull_tendon" width="0.01">
15        <site site="anchor"/>
16        <geom geom="pulley_base" type="wrap"/>
17      </spatial>
18    </tendon>
19
20    <actuator>
21      <position name="servo_1" joint="hinge1"/>
22      <motor name="torque_1" joint="hinge1" gear="5"/>
23    </actuator>
24  </mujoco>

```

Listing 3.3: 执行器与腱绳配置示例

3.11 建模专家建议：数值稳定性调优

1. **抑制振荡**：如果你发现 `position` 执行器让模型像弹簧一样抖动，首先检查 `kp` 是否过大，其次在 `<joint>` 标签中增加 `damping`。
2. **力矩截断**：始终为执行器设置 `ctrlrange`，以防止控制算法输出无穷大的力导致仿真“炸开”（NaN 报错）。
3. **齿轮比的影响**：`gear` 不仅放大力矩，还会反向缩小位置/速度反馈的范围，在编写 C/Python 控制器时需注意这一比例。

3.12 建模最佳实践与避坑指南

1. **质量比例 (Mass Ratio)**：避免让一个 100kg 的箱子骑在一个 0.01kg 的轮子上。这会导致惯性矩阵病态，求解器会报 “Simulation Instability”。建议质量比控制在 1:100 以内。
2. **软约束参数 (solref)**：如果你发现物体碰撞后一直在地板上跳，增加 `solref` 的第一个参数（时间常数）。
3. **可视化调试**：使用模拟器的 `Contact Force` 视图。如果箭头过大且闪烁，说明碰撞刚度设置过高。

3.13 示例代码：完整模型结构

```

1 <mujoco model="comprehensive_example">
2   <compiler angle="degree" coordinate="local" meshdir="assets/" />
3
4   <default>
5     <geom contype="1" conaffinity="1" condim="3" rgba="0.8 0.6 .4 1" />
6     <joint armature="0.01" damping="0.1" />
7   </default>
8
9   <asset>
10    <mesh name="robot_arm" file="arm.stl" />
11    <material name="metal" rgba="0.5 0.5 0.5 1" />
12  </asset>
13
14  <worldbody>
15    <light pos="0 0 5" dir="0 0 -1" />
16    <geom type="plane" size="10 10 0.1" material="metal" />
17    <body name="base" pos="0 0 0.1">
18      <joint type="hinge" axis="0 0 1" />
19      <geom type="mesh" mesh="robot_arm" />
20    </body>
21  </worldbody>
22
23  <actuator>
24    <motor joint="joint1" ctrllrange="-10 10" ctrllimited="true" />
25  </actuator>
26 </mujoco>

```

Listing 3.4: 完整的 MJCF 结构示例

3.14 建模限制与优化建议

1. **质量比例：**系统内最大质量与最小质量比不应超过 10^4 ，否则会导致惯性矩阵病态。
2. **软约束参数：**合理设置 `solref` 中的时间常数。过小会导致高频振荡，过大会导致模型“软绵绵”。
3. **凸性要求：**尽量将复杂的网格拆分为多个凸包几何体，这能显著降低碰撞检测的 CPU 占用。

Chapter 4

XML 引用指南 (XML Reference)

4.1 MJCF 根元素与全局配置

<mujoco> 根标签 所有 MJCF 文件必须以 **<mujoco>** 开始。

- **model**: 模型名称。

<compiler> 编译器选项 控制编译过程中的几何计算和路径解析。

Table 4.1: Compiler 主要属性

属性	描述
boundmass	限制所有物体的最小质量。
inertiafromgeom	是否根据几何体自动计算惯性 (true/false/explicit)。
coordinate	坐标参考系 (local 或 global)。
angle	角度单位 (degree 或 radian)。
meshdir	Mesh 文件的搜索目录路径。

<option> 物理参数 控制物理引擎的全局数值行为。

Table 4.2: Option 核心物理属性

属性	描述
timestep	集成步长 (默认 0.002s)。
gravity	三轴重力矢量, 默认为 [0 0 -9.81]。
integrator	数值集成器 (Euler 或 RK4)。
solver	约束求解器算法 (PGS, CG, Newton)。
cone	摩擦锥类型 (pyramidal 或 elliptic)。
density	介质密度, 用于计算流体阻力。

4.2 世界体与动力学组件

`<worldbody>` 世界体 所有物体的根节点。不接受 `pos` 或 `quat`，因为它固定在世界原点。

`<body>` 刚体

- `name`: 名称。
- `pos`: 相对于父节点的平移位置。
- `quat/euler`: 旋转方向（建议使用四元数）。
- `user`: 用户自定义数据。

`<joint>` 关节 定义运动自由度。

Table 4.3: Joint 属性详解

属性	描述
<code>type</code>	关节类型: free, ball, slide, hinge。
<code>axis</code>	运动轴线方向（对 slide 和 hinge 有效）。
<code>range</code>	关节运动限位（硬约束）。
<code>stiffness</code>	关节弹簧刚度。
<code>damping</code>	关节阻尼。
<code>armature</code>	关节转子惯量。

`<geom>` 几何体

- `type`: plane, sphere, capsule, box, cylinder, mesh。
- `size`: 尺寸参数（随类型变化）。
- `material`: 引用的材质名称。
- `contype/conaffinity`: 碰撞掩码（Bitmasks）。

4.3 资产与驱动系统

`<asset>` 资源定义

- `<mesh>`: 引用 STL/OBJ。其 `scale` 属性可统一缩放。
- `<material>`: 控制外观属性，如 `shininess`, `reflectance`。
- `<texture>`: 支持加载图像，用于 skybox 或物体贴图。

Table 4.4: 执行器类型对比

元素	描述
<code><motor></code>	直接产生广义力矩。
<code><position></code>	具有比例增益的伺服控制器。
<code><velocity></code>	速度控制器。
<code><muscle></code>	具有生物动力学特性的肌肉执行器。

`<actuator>` 执行器

`<tendon>` 腱绳

- `fixed`: 由多个关节线性组合定义的腱。
- `spatial`: 穿过多个三维站点的空间腱绳。

4.4 传感器与约束 (Sensors and Equality)

`<sensor>` 传感器 包含 `accelerometer`, `gyro`, `force`, `torque`, `touch`, `rangefinder` 等。

`<equality>` 等式约束 用于强制执行超越运动链的约束。

- `connect`: 强制两点在世界坐标系中重合。
- `weld`: 强制两个物体保持相对位姿不变（刚性焊接）。

4.5 高级设置

`<custom>` 用户自定义 允许用户在 XML 中定义自定义的常量 (`numeric`) 或字符串 (`text`)，可在 C API 中通过 `mjModel` 读取。

`<visual>` 渲染设置 配置全局阴影、灯光、雾气以及 GUI 中的渲染属性。

Chapter 5

MJCF 实战：从零撰写规范模型

撰写一个高质量的 MJCF 模型文件并非简单的堆砌标签，而是一个从“物理骨架”到“驱动系统”的结构化过程。本章将指导你如何按照工业标准撰写一个规范的模型。

5.1 撰写流水线：四步建模法

一个标准的 MJCF 文件通常遵循以下撰写顺序：

1. 配置层 (Compiler/Option)：设定物理规则（单位、步长、重力）。
2. 资源层 (Asset/Default)：预载模型材质和统一样式（类似 CSS）。
3. 实体层 (Worldbody)：搭建由 Body、Joint、Geom 组成的运动链。
4. 功能层 (Actuator/Sensor)：添加马达驱动和传感器感知。

5.2 第一步：建立“物理环境”与“默认样式”

在空白文件的顶部，首先要确定你的计算基准。通过 `<default>` 标签，我们可以避免在后面为每个零件重复填写参数。

```
1 <mujoco model="my_first_model">
2   <compiler angle="degree" coordinate="local" inertiafromgeom="true"/>
3   <option timestep="0.002" gravity="0 0 -9.81"/>
4
5   <default>
6     <joint damping="0.5" armature="0.01"/>
7     <geom cone="elliptic" friction="1 0.005 0.0001"/>
8   </default>
9 </mujoco>
```

Listing 5.1: 初始化配置

5.3 第二步：构建层级化的“运动链”

在 `<worldbody>` 中，你需要遵循“父子嵌套”逻辑。**记住：子 Body 的坐标是相对于父 Body 的。**

```

1 <worldbody>
2   <light pos="0 0 3" dir="0 0 -1"/>
3   <geom name="floor" type="plane" size="5 5 0.1"/>
4
5   <body name="base_link" pos="0 0 0.1">
6     <geom type="box" size="0.1 0.1 0.1" rgba="0 1 0 1"/>
7
8     <body name="arm_link" pos="0 0 0.1">
9       <joint name="shoulder" type="hinge" axis="0 1 0" range="-90 90"/>
10      <geom type="capsule" fromto="0 0 0 0 0 0.5" size="0.05"/>
11    </body>
12  </body>
13 </worldbody>

```

Listing 5.2: 构建运动链

5.4 第三步：赋予动力 (Actuators)

模型搭好后是“死”的，需要执行器来控制。建议初学者先使用 `position` 类型，因为它最直观。

```

1 <actuator>
2   <position name="servo_shoulder" joint="shoulder" kp="100" ctrlrange="-1.5
3   1.5"/>
4 </actuator>

```

5.5 撰写时的核心原则 (Golden Rules)

5.5.1 1. 几何体优先原则

始终优先使用 `capsule` 和 `sphere`。如果你需要复杂的形状，尝试用多个简单的 `Geom` 组合在一个 `Body` 下，而不是直接导入复杂的 `Mesh`。

5.5.2 2. 命名规范化

为每一个关键的 `joint` 和 `body` 命名。这样在 Python 中，你可以直接通过 `data.joint('shoulder').qpos` 获取数据，而不是去查难记的 ID 索引。

5.5.3 3. 增量式开发

不要一次性写完几百行代码。

- 先写 `worldbody`, 用 MuJoCo Viewer 打开, 检查物体位置对不对。
- 再加 `joint`, 检查关节旋转轴方向 (`axis`) 对不对。
- 最后才加 `actuator` 调试控制逻辑。

5.6 调试技巧: Viewer 是你最好的朋友

当你撰写 MJCF 时, 保持 MuJoCo Viewer 开启。每保存一次 XML, 在 Viewer 中按 `Ctrl+L` 重新载入。

- 如果物体重叠, 你会看到它们像被弹簧弹开一样剧烈抖动。
- 如果关节轴不对, 点击菜单栏的 `Render -> Frame`, 观察坐标轴的指向。

Chapter 6

编程接口深度指南 (Programming Interface)

6.1 MuJoCo 编程哲学：设计与性能

MuJoCo 的 C API 旨在为高性能机器人模拟和机器学习采样提供底层支持。其核心设计理念包括：

- 1. 零动态分配运行 (Zero Dynamic Allocation)** 在核心模拟循环内（即 `mj_step` 调用期间），MuJoCo 保证不会调用任何动态内存分配函数（如 `malloc`）。所有需要的内存都在 `mj_makeData` 阶段预先分配完成。这种设计消除了内存碎片和不确定的延迟。
- 2. 模型与状态完全分离** MuJoCo 严格区分静态模型描述（`mjModel`）和动态状态数据（`mjData`）：
 - **mjModel**: 存储不随时间改变的常数，如刚体的质量、几何体形状、执行器增益。它是只读的，可以被多个线程共享。
 - **mjData**: 存储所有时间相关的变量，如广义位置 *qpos*、广义速度 *qvel*、雅可比矩阵以及传感器读数。每个线程必须拥有独立的 `mjData`。

6.2 核心数据结构详解

6.2.1 mjModel：静态模型定义

`mjModel` 的成员通常以数组形式存储。为了避免报错，正文中引用成员名时需转义下划线。

Table 6.1: `mjModel` 关键成员变量表

变量名	描述与物理意义
<code>nq, nv</code>	广义位置和广义速度的维度。
<code>nu, na</code>	执行器输入信号维度和内部激活量维度。
<code>body_pos</code>	刚体相对于父节点的初始相对位置 ($nbody \times 3$)。

<code>body_quat</code>	刚体相对于父节点的初始四元数方向 ($nbody \times 4$)。
<code>jnt_type</code>	关节类型数组 (0:free, 1:ball, 2:slide, 3:hinge)。
<code>geom_rgba</code>	几何体的颜色与透明度设置 ($ngeom \times 4$)。
<code>actuator_gainprm</code>	执行器增益参数 (如 PID 的 P 增益)。

6.2.2 mjData: 动态模拟数据

`mjData` 是计算最密集的结构体。

Table 6.2: `mjData` 动态状态变量表

变量名	描述与物理意义
<code>time</code>	当前模拟的时间戳 (单位: 秒)。
<code>qpos</code>	广义位置向量 ($nq \times 1$)。
<code>qvel</code>	广义速度向量 ($nv \times 1$)。
<code>qacc</code>	广义加速度向量 ($nv \times 1$)。
<code>ctrl</code>	用户提供的控制输入 ($nu \times 1$)。
<code>xpos</code>	刚体在世界坐标系下的绝对位置 ($nbody \times 3$)。
<code>sensordata</code>	传感器输出数据的扁平缓冲区。
<code>qM</code>	压缩格式存储的对称正定惯性矩阵。

6.3 C API 计算管线: `mj_step` 的深度拆解

`mj_step` 是引擎最顶层的函数, 它内部依次调用了以下子模块。开发者可以通过手动调用这些子模块来实现非标准模拟 (如逆动力学分析)。

6.3.1 1. 位置级别函数 (Position-level)

- `mj_forwardPosition`: 计算前向运动学。更新 `xpos`、`xmat` 等世界坐标系下的量。
- `mj_collision`: 执行碰撞检测, 填充 `mjData->contact` 数组。
- `mj_makeConstraint`: 构造约束雅可比矩阵 J 。

6.3.2 2. 速度级别函数 (Velocity-level)

- `mj_forwardVelocity`: 计算与速度相关的偏置力 (科氏力和离心力)。
- `mj_passive`: 计算重力以外的被动力 (如流体阻力、内部弹簧力)。

6.3.3 3. 力学与集成 (Force & Integration)

- `mj_solve`: 核心步骤, 使用求解器 (PGS/CG/Newton) 求解约束力。
- `mj_integratePos`: 根据速度更新位置 $\mathbf{q} = \mathbf{q} + \Delta t \cdot \dot{\mathbf{q}}$ 。

6.4 多线程并发与性能优化

6.4.1 线程安全策略

由于 `mjModel` 是只读的，在多线程环境（如强化学习的多环境并行采样）中，应遵循：

1. 为每个线程分配一个独立的 `mjData` 对象。
2. 所有线程共享同一个指向 `mjModel` 的指针。
3. 使用 `mj_cloneData` 函数快速复制初始状态。

6.4.2 内存池管理 (Arena Memory)

每个 `mjData` 内部包含一个名为 `arena` 的大块内存缓冲区。所有的临时计算结果（如接触流形）都存储在这里。如果模拟模型极其复杂，用户可能需要在 XML 中增加 `<option totalarena="...">` 的大小。

6.5 超长程序示例：高频率采样控制循环

该示例展示了如何在 C 代码中手动干预模拟管线，并实现自定义的回调控制逻辑。

```

1 #include <stdio.h>
2 #include <string.h>
3 #include "mujoco/mujoco.h"
4
5 // 错误处理缓冲区
6 char error_buffer[1000];
7
8 // 全局指针
9 mjModel* m = NULL;
10 mjData* d = NULL;
11
12 // 定义自定义控制回调函数
13 void my_feedback_controller(const mjModel* m, mjData* d) {
14     // 假设我们要实现一个简单的 PD 控制
15     // qpos[7] 通常是某个关节的位置
16     double target = 0.5;
17     double kp = 100.0;
18     double kv = 10.0;
19
20     // 计算第一个驱动器的控制量
21     if (m->nu > 0) {
22         double error = target - d->qpos[7];
23         double velocity = d->qvel[6];
24         d->ctrl[0] = kp * error - kv * velocity;
25     }
26 }
```

```

26 }
27
28 int main() {
29     // 1. 加载模型
30     m = mj_loadXML("model.xml", NULL, error_buffer, 1000);
31     if (!m) {
32         printf("Error: %s\n", error_buffer);
33         return 1;
34     }
35
36     // 2. 创建数据
37     d = mj_makeData(m);
38
39     // 3. 注册控制回调
40     mjcb_control = my_feedback_controller;
41
42     // 4. 模拟主循环
43     printf("Starting simulation loop...\n");
44     while (d->time < 5.0) {
45         // mj_step 内部会自动调用 mjcb_control
46         mj_step(m, d);
47
48         // 每隔 0.1s 打印一次状态
49         if ((int)(d->time * 1000) % 100 == 0) {
50             printf("Time: %.3f | Control Input[0]: %.4f\n", d->time, d->ctrl[0])
51         }
52     }
53
54     // 5. 释放资源
55     mj_deleteData(d);
56     mj_deleteModel(m);
57
58     return 0;
59 }

```

Listing 6.1: MuJoCo 进阶编程示例：带自定义反馈的模拟循环

6.6 常用 API 函数分类速查

Table 6.3: MuJoCo 常用 C 函数列表

函数名	功能描述
-----	------

<code>mj_loadXML</code>	从 XML 文件编译并加载模型。
<code>mj_step</code>	执行一个完整的模拟步进（包含所有物理计算）。
<code>mj_forward</code>	执行所有前向计算（运动学、力学），但不进行时间积分。
<code>mj_resetData</code>	将模拟状态重置为模型定义的初始位姿。
<code>mj_name2id</code>	根据 XML 中定义的名称获取对应的索引 ID。
<code>mj_id2name</code>	根据索引 ID 获取对象名称。
<code>mj_ray</code>	执行射线检测（用于激光雷达仿真）。

6.7 编程中的常见陷阱与调试技巧

- **NaN 崩溃：**如果 `mjData->qpos` 中出现 NaN，通常是因为 `timestep` 太大或力矩过载。建议使用 `mj_isPyfinite` 检查数值稳定性。
- **四元数归一化：**手动修改 `qpos` 中的旋转分量后，必须调用 `mj_normalizeQuat`，否则会导致旋转矩阵计算异常。
- **数组索引：**记住 MuJoCo 使用扁平化存储，位置数组通常是 nq 维，速度数组是 nv 维。在含有球关节的模型中，这两者维度不相等。

Chapter 7

Python 编程接口 (Python Bindings)

7.1 为什么选择 Python 接口？

MuJoCo 的 Python 绑定不仅是对 C API 的简单封装，它利用了 Python 的动态特性，使得数据访问更加直观。

- **自动内存管理：**不再需要手动调用 `mj_deleteData`。
- **NumPy 集成：**所有的向量和矩阵（如 `qpos`, `qvel`）都直接映射为 NumPy 数组，可以无缝连接到 PyTorch 或 TensorFlow。
- **切片访问：**支持 Python 风格的切片操作，如 `data.qpos[0:7]`。

7.2 基础核心：加载与步进

7.2.1 加载模型

在 Python 中，加载模型仅需一行代码。MuJoCo 会自动处理 XML 的解析与内存分配。

```
1 import mujoco
2 import numpy as np
3
4 # 1. 加载模型（支持从文件或字符串加载）
5 model = mujoco.MjModel.from_xml_path("humanoid.xml")
6 data = mujoco.MjData(model)
7
8 # 2. 查看模型基本信息
9 print(f"自由度数量: {model.nv}")
10 print(f"关节名称: {[model.joint(i).name for i in range(model.njnt)]}")
```

Listing 7.1: Python 基础加载流程

7.3 数据交互：读取与控制

7.3.1 状态读取 (Getter)

MuJoCo Python 接口将数组封装成了属性，访问非常自然：

- 位置：`data.qpos` —— 对应 n_q 维数组。
- 速度：`data.qvel` —— 对应 n_v 维数组。
- 时间：`data.time` —— 当前仿真时间。

7.3.2 控制输入 (Setter)

给机器人施加力矩，只需修改 `data.ctrl` 数组：

```
1 # 给所有电机施加 1.0 的控制信号
2 data.ctrl[:] = 1.0
3 mujoco.mj_step(model, data)
```

7.4 高级特性：命名访问 (Named Access)

在 C 语言中，你需要通过 `mj_name2id` 查索引，但在 Python 中，MuJoCo 提供了更优雅的命名访问器：

```
1 # 获取名为 'wrist_joint' 的关节位置
2 wrist_pos = data.joint('wrist_joint').qpos
3
4 # 获取名为 'sensor_touch' 的传感器数据
5 touch_force = data.sensor('sensor_touch').data
6
7 # 获取名为 'torso' 的刚体世界坐标
8 torso_xpos = data.body('torso').xpos
```

Listing 7.2: 使用名称直接访问对象

7.5 零基础实战：手把手构建一个仿真环境

这个例子展示了如何从零开始：加载、初始化、循环、并获取传感器数据。

```
1 import mujoco
2 import time
3
4 # 1. 准备模型 这里直接用字符串定义一个简单的单摆()
5 xml_content = """
6 <mujoco>
7   <worldbody>
```



```

8     <light pos="0 0 3"/>
9     <geom type="plane" size="1 1 0.1"/>
10    <body pos="0 0 1">
11        <joint name="swing" type="hinge" axis="0 1 0"/>
12        <geom type="capsule" size="0.05" fromto="0 0 0 0 0 -0.5" rgba="1 0 0 1"/>
13    </body>
14 </worldbody>
15 </mujoco>
16 """
17 model = mujoco.MjModel.from_xml_string(xml_content)
18 data = mujoco.MjData(model)
19
20 # 2. 设置初始状态
21 data.joint('swing').qpos = 0.5 # 初始偏角
22
23 # 3. 开启仿真循环
24 print("开始 Python 仿真...")
25 for _ in range(1000):
26     # 模拟简单的重力补偿: 给关节一个反向的力
27     current_vel = data.joint('swing').qvel
28     data.ctrl[0] = -0.1 * current_vel
29
30     # 物理步进
31     mujoco.mj_step(model, data)
32
33     if _ % 100 == 0:
34         print(f"Time: {data.time:.2f} | Joint Angle: {data.qpos[0]:.4f}")
35
36     # 控制仿真速度, 使其看起来像实时 (可选)
37     # time.sleep(model.opt.timestep)

```

Listing 7.3: 完整的 Python 仿真循环示例

7.6 可视化渲染 (Rendering)

Python 接口集成了 `mujoco.viewer`, 这是调试模型最强大的工具。

```

1 import mujoco.viewer
2
3 # 启动一个交互式窗口, 你可以用鼠标拖拽机器人
4 with mujoco.viewer.launch_passive(model, data) as viewer:
5     while viewer.is_running():
6         mujoco.mj_step(model, data)
7         viewer.sync() # 同步渲染器状态

```

Listing 7.4: 开启交互式可视化窗口

7.7 数值稳定性与性能调优 (给新手的建议)

1. **避开循环操作数组：**如果你想修改 100 个关节的力，请使用 `data.ctrl[:] = np.array(...)` 而不是写 `for` 循环。NumPy 的批量操作比 Python 循环快数百倍。
2. **注意副本问题：**`data.qpos` 返回的是原始内存的 **** 视图 (View)****。如果你执行 `my_pos = data.qpos`，修改 `my_pos` 会直接改变仿真状态。如果你想保存快照，请使用 `my_pos = data.qpos.copy()`。
3. **多线程：**Python 有全局解释器锁 (GIL)，但在执行 `mj_step` 时，MuJoCo 会释放 GIL。这意味着你可以利用 Python 的 `threading` 库来并行跑多个模拟环境。

7.8 常用 API 速查表 (Python 版)

Table 7.1: Python 常用方法

方法名	功能描述
<code>mujoco.MjModel.from_xml_path</code>	从文件加载模型。
<code>mujoco.mj_step(m, d)</code>	核心物理步进。
<code>mujoco.mj_forward(m, d)</code>	更新所有位置相关的中间变量（不积分时间）。
<code>data.joint('name')</code>	通过名字访问关节对象。
<code>data.sensor('name').data</code>	读取特定传感器的数值数组。
<code>mujoco.mj_resetData(m, d)</code>	重置所有数据到初始状态。

Chapter 8

MJX: GPU 加速仿真 (MuJoCo on JAX)

8.1 什么是 MJX?——从“单兵作战”到“军团冲锋”

传统的 MuJoCo (C/Python) 是在 CPU 上运行的，虽然很快，但一次只能算一个环境。**MJX** 是 MuJoCo 的新一代版本，它使用 Google 的 JAX 框架重写了核心算法：

- 万倍加速：利用 GPU 的并行能力，同时仿真数万个机器人。
- 端到端可微：整个物理引擎变为一个巨大的数学函数，可以直接对物理参数求导。
- 消除数据传输瓶颈：物理仿真和深度学习模型（如神经网络）都在 GPU 内存中，无需在 CPU 和 GPU 之间搬运数据。

8.2 MJX 的核心概念：向量化 (Vectorization)

在 Python 版中，我们写循环来跑仿真；在 MJX 中，我们使用 **Vmap**：

- 数据批处理：`mjx.data` 的维度通常是 (batch, nq)，其中 batch 可能是 4096。
- 无分叉逻辑：为了在 GPU 上跑得快，MJX 尽量避免了“如果... 就...”这种判断，而是使用掩码计算。

8.3 快速入门：MJX 编程三部曲

对于零基础用户，MJX 的操作逻辑与 Python 版高度相似，但多了一个“编译”步骤。

```
1 import jax
2 from jax import numpy as jp
3 from mujoco import mjx
4 import mujoco
5
6 # 1. 依然使用原生的 MuJoCo 加载 XML
```

```

7 model = mujoco.MjModel.from_xml_path("robot.xml")
8 data = mujoco.MjData(model)
9
10 # 2. 将模型和数据“传送”到 GPU
11 mjx_model = mjx.put_model(model)
12 mjx_data = mjx.put_data(model, data)
13
14 # 3. 定义一个并行的步进函数
15 @jax.jit # 关键: 使用 JIT 编译成高速显卡指令
16 def step(m, d):
17     return mjx.step(m, d)
18
19 # 现在, 这个 step 函数在 GPU 上运行, 速度极快!
20 mjx_data = step(mjx_model, mjx_data)

```

Listing 8.1: MJX 基础运行流程

8.4 深度解析: MJX 与原生 MuJoCo 的差异

为了在 GPU 上实现并行, MJX 在底层做了一些权衡:

Table 8.1: MJX 与原生 MuJoCo 对比

特性	MJX 表现
Solver	默认使用 CG (共轭梯度) 或 Newton , 因为它们在 GPU 上比 PGS 更容易并行。
Precision	建议使用 float32 以获得最高速度, 但也支持 float64 。
Custom Code	不支持 C 语言回调函数, 必须用 JAX 重写控制逻辑。
Collision	支持几乎所有几何体, 但对于超复杂 Mesh 的处理略有简化。

8.5 零基础实战: 大规模并行采样

想象一下, 你想让 2048 个机器人同时尝试不同的控制力, 看谁跳得高:

```

1 # 假设我们有 2048 个不同的随机控制信号
2 batch_size = 2048
3 ctrl_signals = jax.random.uniform(key, (batch_size, model.nu))
4
5 # 使用 jax.vmap 自动将“一个环境”的逻辑扩展到“两千个”
6 batched_step = jax.vmap(mjx.step, in_axes=(None, 0))
7
8 # 只需要这一行, GPU 就会同时计算 2048 个机器人的下一帧!

```

```
9 mjax_data = batched_step(mjax_model, mjax_data.replace(ctrl=ctrl_signals))
```

Listing 8.2: 2048 个环境并行仿真示例

8.6 给新手的避坑指南

8.6.1 1. “冷启动”等待

第一次运行 MJX 代码时，你会发现程序卡住几秒钟。这是 JAX 正在将 Python 代码编译为 GPU 代码。一旦编译完成，后面的运行将是闪电般的。

8.6.2 2. 静态与动态的界限

在 JAX 中，所有的变量都应该是 `jp.array`（不可变的）。如果你习惯了像 C 语言那样直接修改 `data.qpos[0] = 1.0`，在 MJX 中你需要改写为：`new_data = data.replace(qpos=new_qpos)`。

8.6.3 3. 内存显存限制

虽然 GPU 能跑数万个环境，但如果你的模型特别复杂（比如带皮肤的人体模型），显存可能会溢出。建议从 128 个并行环境开始尝试。

8.7 MJX 的典型应用场景

1. **强化学习 (RL)**：如训练四足机器人走动，MJX 能将训练时间从几天缩短到几分钟。
2. **进化算法**：同时生成数千种机器人外形，筛选出跑得最快的。
3. **蒙特卡洛采样**：预测机器人受到不同方向撞击后的倒地概率。

